

Algorithm state: (C, L, R, W) :

- $C : \text{Threads} \rightarrow VC$. $C_t = C(t)$. C_t represents what thread t knows about the logical clocks of all threads. If I am thread t , then: $C_t(t)$ is my logical clock. It will always be **positive**. For $u \neq t$, $C_t(u) = z$ means “I know that thread u ’s logical clock is at least z ”. When t **acquires a lock**, t gets information about the logical clocks of threads who previously held the lock.
- $L : \text{Locks} \rightarrow VC$. $L_m = L(m)$. L_m represents the value of the vector clock associated with the last thread that released m , when the release occurred. For a lock m and thread t : $L_m(t) = z$ means that the last thread to release m knew that thread t ’s logical clock was at least z when it released the mutex.
- $R : \text{Locations} \rightarrow VC$. $R_x = R(x)$. $R_x(t)$ represents the value of thread t ’s logical clock when it last read from x . For a location x and thread t : $R_x(t) = 0$ means that t has never read from x . Otherwise $R_x(t)$ was t ’s logical clock last time t read from x .
- $W : \text{Locations} \rightarrow VC$. Similar to R .